

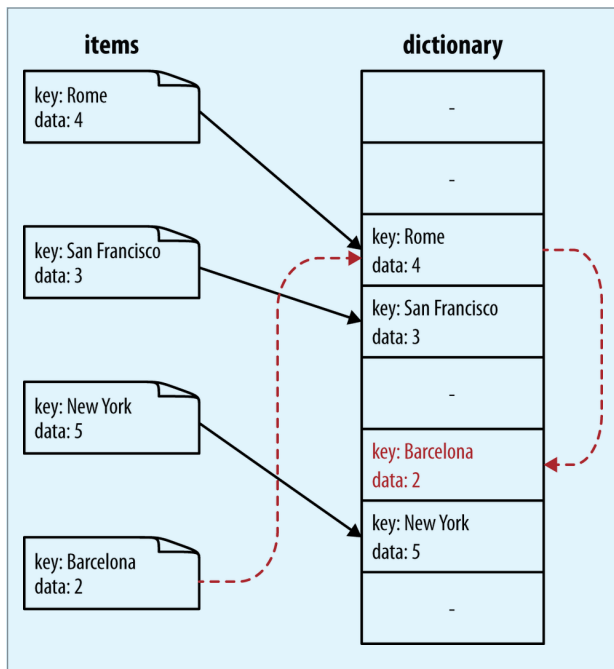


Figure 2: Hash table with collision (Credits: <https://www.oreilly.com/library/view/high-performance-python/9781449361747/ch04.html>)

Although the hash collision and the adjustment of the size of the hash table will slow down the speed, this happens very rarely. Therefore, on average, this can still ensure that the time complexity of insert, find, and delete is $O(1)$.

Dicts have significant memory overhead. Even an empty dictionary occupies more memory than lists or tuples. Here's a sample:

```
import sys

sample_dict = {}
print(sample_dict, " size of empty dict", sys.
sizeof(sample_dict), type(sample_dict))

sample_list = list()
print(sample_list, " size of empty list", sys.
sizeof(sample_list), type(sample_list))
```

Output:

```
{ } size of empty dict 64 <class 'dict'>
[] size of empty list 56 <class 'list'>
```

Though dicts require additional storage they ensure better performance. We can optimise them and this is a continuous process. One just needs to remember that optimisation is the altar where maintainability is sacrificed. Modifying the dict while iterating is not a good idea.

Set and frozen set types are also implemented with a hash table, except that each bucket holds only a reference to the element. If there is a slight variation in dict algorithms, we need to perform lookups, whereas sets are used to save unique objects.

As mentioned earlier, dictionaries and sets outperform lists and tuples, especially when it comes to search, addition, and deletion operations. Dictionaries utilise hash tables to achieve constant time complexity, making them highly efficient. Sets and dictionaries share similarities, with the only difference being that sets do not have key-value pairs and consist of unique and unordered element combinations.

Lastly, let's give a round of applause to Python's core developers, researchers, and other engineers who work behind the scenes to improve Python's efficiency, security, and overall performance. **END** 🐧

References

- Source code: dictobject.c
- https://nolongerset.com/content/images/2020/12/key-5105878_1920.jpg
- <https://github.com/python/cpython/blob/main/Objects/dictobject.c>
- <https://stackoverflow.com/questions/52023758/list-vs-dictionary-vs-set-for-finding-the-number-of-times-a-number-appears>
- <https://towardsdatascience.com/faster-lookups-in-python-1d7503e9cd38>
- <https://dzone.com/articles/python-memo-2-dictionary-vs-set-1>
- <https://www.oreilly.com/library/view/high-performance-python/9781449361747/ch04.html>
- <https://tenthousandmeters.com/blog/python-behind-the-scenes-10-how-python-dictionaries-work/>
- <https://realpython.com/python-hash-table/>
- <https://www.acunetix.com/vulnerabilities/web/php-hash-collision-denial-of-service-vulnerability/>
- <https://www.asmeurer.com/blog/posts/what-happens-when-you-mess-with-hashing-in-python/>
- <https://www.youtube.com/watch?v=npw4s1QTmPg>
- https://www.geeksforgeeks.org/*/
- <https://morepypy.blogspot.com/2015/01/faster-more-memory-efficient-and-more.html>
- <https://mail.python.org/pipermail/python-dev/2012-December/123028.html>
- <https://www.fluentpython.com/extra/internals-of-sets-and-dicts/>
- <https://www.youtube.com/watch?v=66P5FMkWoVU&t=56s>

By: Thangaselvi Arichandrapandian

The author is a lifelong learner influenced by the quote:
"The more I learn, the more I realise how much I don't know."
 -Albert Einstein